

```
In [11]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# =====
# 1. LOAD THE DATA (Safely!)
# =====
print("Loading data...")
# We use nrows=500000 to avoid crashing your RAM.
# Once you know your code works, you can try loading more, or use a tool like 'Dask'
df = pd.read_csv(r'C:\Users\LOQ\Downloads\US_Accidents_March23.csv\US_Accidents_Mar

print(f"Dataset loaded successfully with {df.shape[0]} rows and {df.shape[1]} column
df.head()
```

Loading data...

Dataset loaded successfully with 500000 rows and 46 columns.

```
Out[11]:
```

	ID	Source	Severity	Start_Time	End_Time	Start_Lat	Start_Lng	End_Lat	End_Lng
0	A-1	Source2	3	2016-02-08 05:46:00	2016-02-08 11:00:00	39.865147	-84.058723	NaN	NaN
1	A-2	Source2	2	2016-02-08 06:07:59	2016-02-08 06:37:59	39.928059	-82.831184	NaN	NaN
2	A-3	Source2	2	2016-02-08 06:49:27	2016-02-08 07:19:27	39.063148	-84.032608	NaN	NaN
3	A-4	Source2	3	2016-02-08 07:23:34	2016-02-08 07:53:34	39.747753	-84.205582	NaN	NaN
4	A-5	Source2	2	2016-02-08 07:39:07	2016-02-08 08:09:07	39.627781	-84.188354	NaN	NaN

5 rows × 46 columns

```
In [4]: # =====
# 2. DATA CLEANING & PREPARATION
# =====
print("\nCleaning data...")
# Convert the 'Start_Time' column to actual datetime objects so we can extract hour
df['Start_Time'] = pd.to_datetime(df['Start_Time'], errors='coerce')

# Drop rows where critical information is missing to keep our analysis clean
df.dropna(subset=['Start_Time', 'Severity', 'City', 'Weather_Condition'], inplace=T

# Extract the Hour of the day from the Start_Time for temporal analysis
df['Hour'] = df['Start_Time'].dt.hour
print(df.head())
```

Cleaning data...

	ID	Source	Severity	Start_Time	End_Time	Start_Lat	\
0	A-1	Source2	3	2016-02-08 05:46:00	2016-02-08 11:00:00	39.865147	
1	A-2	Source2	2	2016-02-08 06:07:59	2016-02-08 06:37:59	39.928059	
2	A-3	Source2	2	2016-02-08 06:49:27	2016-02-08 07:19:27	39.063148	
3	A-4	Source2	3	2016-02-08 07:23:34	2016-02-08 07:53:34	39.747753	
4	A-5	Source2	2	2016-02-08 07:39:07	2016-02-08 08:09:07	39.627781	

	Start_Lng	End_Lat	End_Lng	Distance(mi)	...	Station	Stop	\
0	-84.058723	NaN	NaN	0.01	...	False	False	
1	-82.831184	NaN	NaN	0.01	...	False	False	
2	-84.032608	NaN	NaN	0.01	...	False	False	
3	-84.205582	NaN	NaN	0.01	...	False	False	
4	-84.188354	NaN	NaN	0.01	...	False	False	

	Traffic_Calming	Traffic_Signal	Turning_Loop	Sunrise_Sunset	Civil_Twilight	\
0	False	False	False	Night	Night	
1	False	False	False	Night	Night	
2	False	True	False	Night	Night	
3	False	False	False	Night	Day	
4	False	True	False	Day	Day	

	Nautical_Twilight	Astronomical_Twilight	Hour
0	Night	Night	5
1	Night	Day	6
2	Day	Day	6
3	Day	Day	7
4	Day	Day	7

[5 rows x 47 columns]

```
In [5]: print(f"Data remaining after cleaning: {df.shape[0]} rows.")
```

Data remaining after cleaning: 489794 rows.

```
In [6]: print(df.info())
```

```

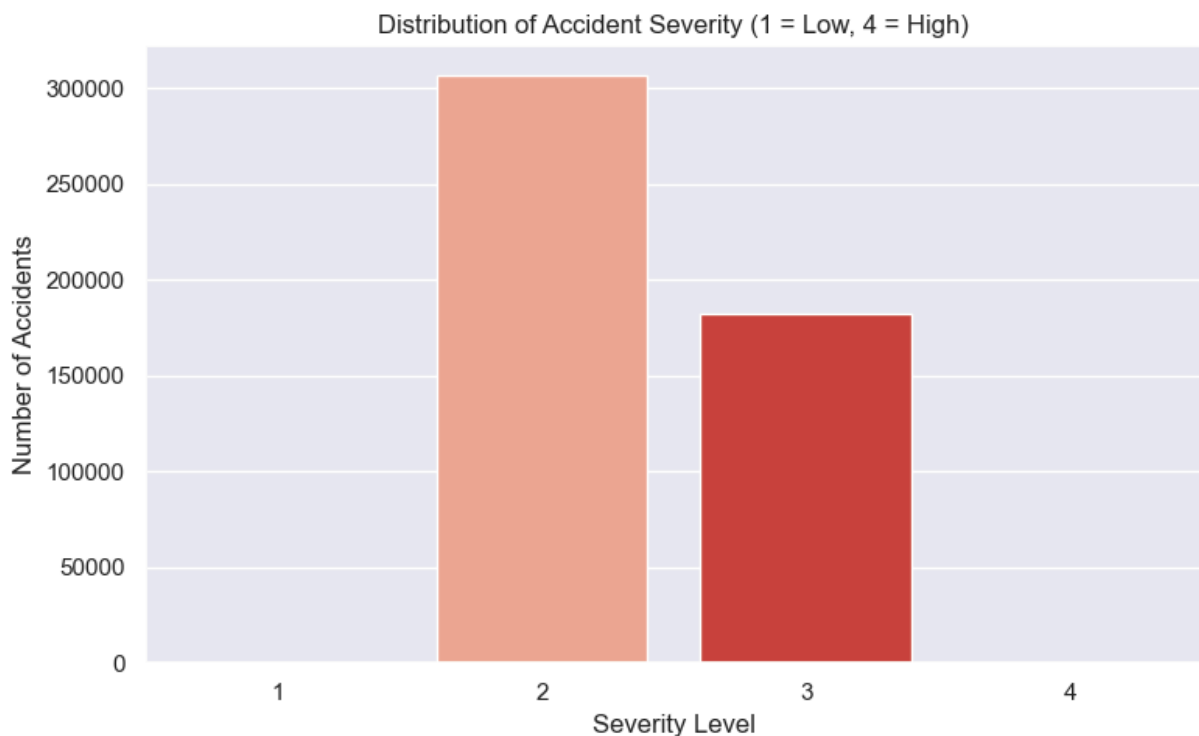
<class 'pandas.core.frame.DataFrame'>
Index: 489794 entries, 0 to 499999
Data columns (total 47 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                    489794 non-null object
1   Source                               489794 non-null object
2   Severity                             489794 non-null int64
3   Start_Time                           489794 non-null datetime64[ns]
4   End_Time                             489794 non-null object
5   Start_Lat                            489794 non-null float64
6   Start_Lng                            489794 non-null float64
7   End_Lat                              0 non-null      float64
8   End_Lng                              0 non-null      float64
9   Distance(mi)                         489794 non-null float64
10  Description                           489794 non-null object
11  Street                               489794 non-null object
12  City                                 489794 non-null object
13  County                              489794 non-null object
14  State                               489794 non-null object
15  Zipcode                             489794 non-null object
16  Country                             489794 non-null object
17  Timezone                            489794 non-null object
18  Airport_Code                        489794 non-null object
19  Weather_Timestamp                   489794 non-null object
20  Temperature(F)                      487942 non-null float64
21  Wind_Chill(F)                       61742 non-null  float64
22  Humidity(%)                         487284 non-null float64
23  Pressure(in)                        489143 non-null float64
24  Visibility(mi)                      487990 non-null float64
25  Wind_Direction                      489767 non-null object
26  Wind_Speed(mph)                     408677 non-null float64
27  Precipitation(in)                   51549 non-null  float64
28  Weather_Condition                   489794 non-null object
29  Amenity                             489794 non-null bool
30  Bump                                489794 non-null bool
31  Crossing                            489794 non-null bool
32  Give_Way                            489794 non-null bool
33  Junction                            489794 non-null bool
34  No_Exit                             489794 non-null bool
35  Railway                             489794 non-null bool
36  Roundabout                          489794 non-null bool
37  Station                             489794 non-null bool
38  Stop                                489794 non-null bool
39  Traffic_Calming                     489794 non-null bool
40  Traffic_Signal                      489794 non-null bool
41  Turning_Loop                        489794 non-null bool
42  Sunrise_Sunset                      489794 non-null object
43  Civil_Twilight                      489794 non-null object
44  Nautical_Twilight                  489794 non-null object
45  Astronomical_Twilight               489794 non-null object
46  Hour                                489794 non-null int32
dtypes: bool(13), datetime64[ns](1), float64(12), int32(1), int64(1), object(19)
memory usage: 135.0+ MB
None

```

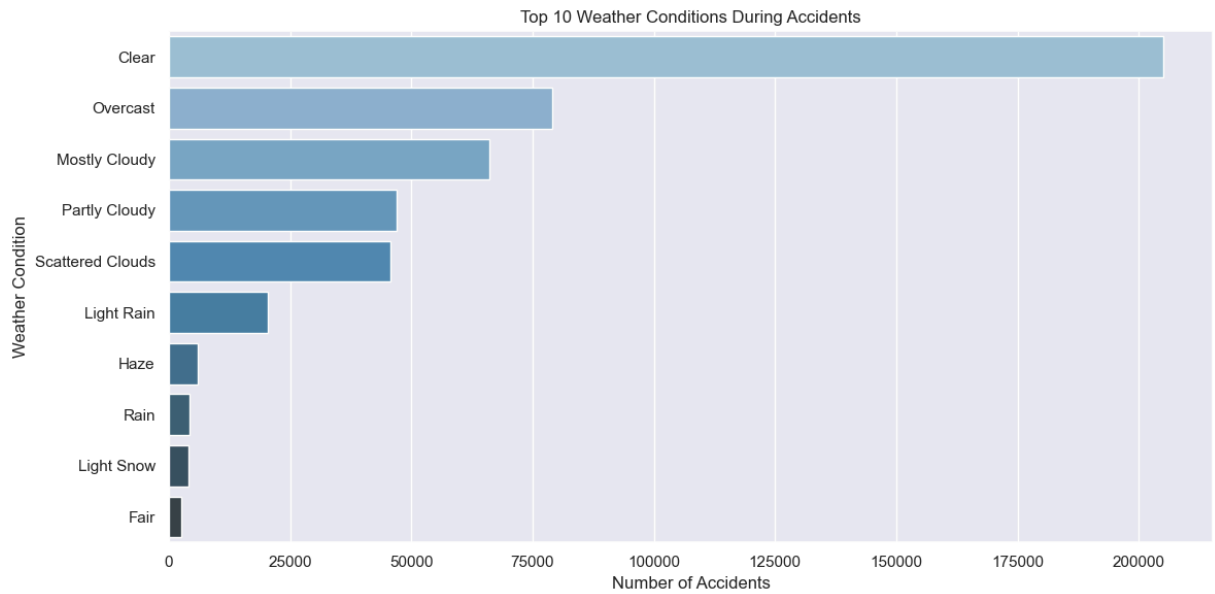
```
In [7]: # 3. EXPLORATORY DATA ANALYSIS & VISUALIZATION

# Set the visual style for all graphs
sns.set_theme(style="darkgrid")

# --- Chart 1: Accident Severity Distribution ---
# Severity is rated 1 to 4 (1 being least impact on traffic, 4 being significant de
plt.figure(figsize=(8, 5))
sns.countplot(x='Severity', data=df, hue='Severity', palette='Reds', legend=False)
plt.title('Distribution of Accident Severity (1 = Low, 4 = High)')
plt.xlabel('Severity Level')
plt.ylabel('Number of Accidents')
plt.tight_layout()
plt.savefig('accident_severity.png')
plt.show()
```



```
In [8]: # --- Chart 3: Top 10 Weather Conditions During Accidents ---
# Does bad weather cause more crashes, or do most happen on clear days?
plt.figure(figsize=(12, 6))
top_weather = df['Weather_Condition'].value_counts().head(10)
sns.barplot(y=top_weather.index, x=top_weather.values, hue=top_weather.index, palet
plt.title('Top 10 Weather Conditions During Accidents')
plt.xlabel('Number of Accidents')
plt.ylabel('Weather Condition')
plt.tight_layout()
plt.savefig('weather_conditions.png')
plt.show()
```



```
In [9]: # --- Chart 4: Top 10 States with the Most Accidents ---
plt.figure(figsize=(10, 6))
top_states = df['State'].value_counts().head(10)
sns.barplot(x=top_states.index, y=top_states.values, hue=top_states.index, palette=
plt.title('Top 10 States with the Highest Number of Accidents')
plt.xlabel('State Code')
plt.ylabel('Total Accidents')
plt.tight_layout()
plt.savefig('top_states.png')
plt.show()
```

